The Inbox Avenger: Empowering Users Through Intelligent Email Management

In the modern digital landscape, email remains the primary medium for professional communication, service registration, and personal correspondence. However, this reliance has led to significant challenges: inbox overload, notification fatigue, the proliferation of subscription lists, and the ever-increasing sophistication of phishing attacks. Recognizing these overlapping issues, "The Inbox Avenger" was conceptualized as a comprehensive digital shield. Developed by Jordan Wishom, Jonathan Cagle, and Benjamin Marshall for ACO 494, the project aimed to return control of digital attention and security to the user. Initially envisioned as a dual-purpose tool combining an AI-assisted inbox cleanup engine with an encrypted password vault, the project ultimately narrowed its focus to deliver a highly interactive, robust Chrome extension. This paper details the design, iterative implementation, testing, and eventual conclusions drawn from building The Inbox Avenger, a tool designed to block noise, mitigate danger, and simplify email management.

The architectural design of The Inbox Avenger underwent a significant evolution as the team balanced ambitious goals with the practical constraints of a single academic semester. The initial design proposed a robust backend architecture relying on Python (Flask/FastAPI), an SQLite local database, and AES-256 encryption for secure credential storage. The intelligent filtering component was designed to utilize supervised machine learning classifiers, such as Logistic Regression or Naive Bayes, trained on datasets to detect spam and phishing.

Central to the system's design was a Risk Scoring Engine based on heuristic weighting. Indicators such as suspicious URL mismatches, urgent language, and mismatched domains were assigned point values to generate an aggregate threat score. While the initial backend-heavy

design provided a strong theoretical foundation, the team recognized that integrating a live ML backend with user inboxes posed massive privacy and scope challenges. Consequently, the design pivoted toward a client-side architecture: a Google Chrome Extension running directly within the browser's Document Object Model (DOM). This design shift ensured that all processing could happen locally on the user's machine, preserving privacy by eliminating the need to transmit sensitive email data to an external server while still providing real-time, interactive categorization.

The implementation of The Inbox Avenger was highly iterative, transitioning from backend concepts to a polished front-end prototype across five distinct versions. The project began with a modular Java-based prototype to validate core object-oriented concepts, including classes for User Authentication, a Password Vault, and a Phishing Detector. Once the logic was verified, the team moved to web technologies to build the Chrome extension.

Version 1, the Lite prototype, was a proof-of-concept script that injected a scan button into the Gmail interface. It scraped visible rows in the DOM, evaluated the text against hardcoded regular expressions for risk and promotional keywords, and forcefully reordered the rows while appending color-coded badges for High Risk, Promo, and Low Priority.

Version 2, the Adaptive prototype, improved usability by introducing a floating settings panel, customizable themes, and scan depth controls. Crucially, it added a local adaptive learning system. Users could click training buttons to let the extension dynamically update its local heuristic memory in Chrome storage, mimicking machine learning through local vocabulary tokenization.

Version 3, the Smart prototype, shifted focus to deep inbox management by introducing a dedicated pop-up HTML interface. It expanded whole-message analysis by scanning opened

emails to hunt for explicit or hidden unsubscribe links, pulling them into a centralized Subscription Center for easy user access.

The final iterations, Versions 4 and 5, pushed the extension to its maximum potential. Version 4 introduced a massive word database, a triage queue, and saved filters. Finally, Version 5 laid the groundwork for transitioning away from DOM-scraping by implementing an API-ready Gmail architecture. This included mailbox indexing scaffolds, advanced search over indexed mail, and scaffolds for creating actual Gmail labels.

Testing and evaluation for The Inbox Avenger heavily prioritized user privacy and heuristic accuracy. Because the tool was designed to avoid transmitting sensitive email data to external servers, all testing was conducted locally using Chrome Developer Mode and simulated inbox datasets rather than live production environments. A primary focus of the evaluation phase was calibrating the Risk Scoring Engine. Since the prototype relied on keyword tokenization and regular expressions rather than a backend neural network, the team manually tested and tuned the heuristic weights to minimize false positives. For example, edge-case testing was conducted to ensure that a legitimate promotional newsletter containing a word like "urgent" was correctly categorized as a "Promo" rather than falsely flagged as a "High Risk" phishing attempt. Furthermore, the "local adaptive learning" feature introduced in Version 2 was rigorously evaluated to confirm that user training clicks successfully updated the local vocabulary cache and accurately applied new weighting to future emails.

Beyond heuristic accuracy, extensive performance and stability testing were required to handle the highly dynamic nature of Gmail’s Document Object Model (DOM). The team evaluated the extension’s Mutation Observers to ensure that injected UI elements, such as color-coded badges, floating panels, and row reordering, rendered seamlessly without crashing

the browser tab as the user scrolled. Performance testing became especially critical in Versions 3 and 4, which were assessed on their ability to swiftly open an email, scrape the body for hidden or explicit unsubscribe links, and populate the Subscription Center without disrupting the native Gmail experience. Finally, the evaluation of Version 5 shifted from visual UI testing toward architectural validation, verifying that the new API-ready scaffolding, mock OAuth flows, and localized search functions operated correctly, proving the system's readiness for a full backend integration.

The Inbox Avenger successfully demonstrated that an automated, locally processed inbox filtering system is both technically feasible and highly valuable to end users. By pivoting to a Chrome extension, the team was able to deliver a tangible, interactive product that directly addressed the problems of email overload and subscription fatigue. The prototype proved that heuristic rules, combined with a well-designed user interface, can effectively triage emails into actionable categories without requiring the user to have deep technical knowledge. The project met its academic and technical goals, yielding a highly functional proof-of-concept that empowers users to clean their inboxes safely and efficiently.

The development cycle of The Inbox Avenger yielded several critical insights. First and foremost was the inherent limitation of DOM manipulation. Because earlier versions relied on reading Gmail’s HTML structure, the extension was inherently brittle; any update by Google to Gmail’s CSS classes or layout could break the scanner. This underscored a vital software engineering lesson: scraping the UI is excellent for rapid prototyping, but unstable for long-term production.

Secondly, the team learned the immense value of iterative development and agile pivoting. Moving from a complex backend vision to a lightweight front-end allowed the team to

finish a working product within a single semester. Finally, the project highlighted the delicate balance between advanced functionality and user privacy. By prioritizing local processing and eventually scaffolding a secure OAuth and Gmail API architecture, the team learned how to architect systems that provide deep data analysis without compromising personal security. Moving forward, the clear next step for the project would be to fully implement the Gmail API integration, transitioning from a localized DOM-scraper to a true, mailbox-wide management platform.